Experiment 6

Aim: Perform sentiment analysis on real-life data

Theory:

Sentiment analysis is a Natural Language Processing (NLP) technique used to identify whether a piece of text expresses a positive, negative, or neutral opinion. It is widely applied to real-life data such as movie reviews, product feedback, and social media posts. Traditional text representation methods like Bag-of-Words and TF-IDF fail to capture semantic relationships between words. To address this limitation, **Word2Vec** was introduced as an advanced word embedding technique that converts words into meaningful dense vector representations.

Word2Vec, developed by Tomas Mikolov at Google, uses shallow neural networks to learn word relationships from large text corpora. It has two main architectures: Continuous Bag of Words (CBOW) and Skip-gram. CBOW predicts a word from its surrounding context, while Skip-gram predicts surrounding words from a target word. Through this training process, Word2Vec places semantically similar words close together in a vector space, allowing models to understand relationships and similarities between words.

In sentiment analysis, text data is first preprocessed through cleaning, tokenization, and normalization. The Word2Vec model then generates vector representations for each word in the dataset. Since machine learning models require fixed-length inputs, document-level vectors are typically created by averaging the word embeddings in a sentence or review. These vectors are then provided to classifiers such as Logistic Regression or Support Vector Machines to predict sentiment.

The major advantage of Word2Vec is its ability to capture semantic meaning and reduce dimensionality compared to traditional encoding methods. Although it does not fully capture word order or complex context, it significantly improves sentiment classification performance and serves as a foundational technique for modern NLP models.

Source Code and Outputs:

[11] ✓ 19s

```
!pip install gensim nltk scikit-learn

import numpy as np
import pandas as pd
import re
import nltk
import gensim
import matplotlib.pyplot as plt

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from gensim.models import Word2Vec

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Embedding
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.preprocessing.text import Tokenizer
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
```

[12] ✓ 0s

```
nltk.download('punkt')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

[13] ✓ 4s

```
from tensorflow.keras.datasets import imdb

# Load dataset (top 10000 most frequent words)
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)

word_index = imdb.get_word_index()
reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

[14] ✓ 6s

```
def decode_review(text):
    return ' '.join([reverse_word_index.get(i - 3, '?') for i in text])

X_train_text = [decode_review(x) for x in X_train]
X_test_text = [decode_review(x) for x in X_test]
```

[15] ✓ 46s

```
nltk.download('punkt_tab')
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z]', ' ', text)
    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words]
    return tokens

X_train_tokens = [preprocess(text) for text in X_train_text]
X_test_tokens = [preprocess(text) for text in X_test_text]
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

[16] ✓ 23s

```python
w2v_model = Word2Vec(
    sentences=X_train_tokens,
    vector_size=100,
    window=5,
    min_count=2,
    workers=4
)
```

[17] ✓ 19s

```python
def get_sentence_vector(tokens):
    vectors = []
    for word in tokens:
        if word in w2v_model.wv:
            vectors.append(w2v_model.wv[word])
    if len(vectors) == 0:
        return np.zeros(100)
    return np.mean(vectors, axis=0)

X_train_vectors = np.array([get_sentence_vector(tokens) for tokens in X_train_tokens])
X_test_vectors = np.array([get_sentence_vector(tokens) for tokens in X_test_tokens])
```

[18] ✓ 0s

```python
model = Sequential()
model.add(Dense(64, activation='relu', input_shape=(100,)))
model.add(Dense(32, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	6,464
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 1)	33

```
Total params: 8,577 (33.50 KB)
Trainable params: 8,577 (33.50 KB)
Non-trainable params: 0 (0.00 B)
```

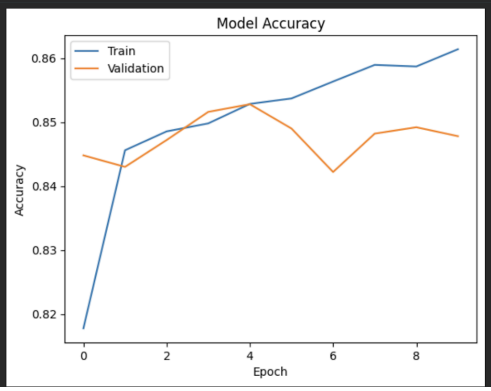
[19] ✓ 10s

```python
history = model.fit(
    X_train_vectors,
    y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2
)
```

```
Epoch 1/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.7738 - loss: 0.4931 - val_accuracy: 0.8448 - val_loss: 0.3633
Epoch 2/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8487 - loss: 0.3628 - val_accuracy: 0.8430 - val_loss: 0.3590
Epoch 3/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8453 - loss: 0.3569 - val_accuracy: 0.8472 - val_loss: 0.3558
Epoch 4/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8465 - loss: 0.3539 - val_accuracy: 0.8516 - val_loss: 0.3497
Epoch 5/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8527 - loss: 0.3383 - val_accuracy: 0.8528 - val_loss: 0.3477
```

```
[20]
✓ 5s
y_pred = model.predict(X_test_vectors)
y_pred = (y_pred > 0.5).astype(int)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

782/782 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step
Accuracy: 0.84692
              precision    recall  f1-score   support

           0       0.88      0.80      0.84     12500
           1       0.82      0.89      0.85     12500

    accuracy                           0.85     25000
   macro avg       0.85      0.85      0.85     25000
weighted avg       0.85      0.85      0.85     25000
```

```
[21]
✓ 0s
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()
```



Learning Outcomes:

I am able to understand and perform sentiment analysis on real-life textual data by preprocessing text, extracting relevant features, and classifying opinions into positive, negative, or neutral categories, thereby enabling meaningful interpretation of user feedback, reviews, and social media data for real-world applications.